# ВКР: как я хочу её делать — коротко

| | |
|---|---|
| Студент | Евстигнеев Никита Артёмович, группа P3431, 09.03.01 «Информатика и вычислительная техника» |
| Тема | «Разработка ИИ-агента для приложений граничного искусственного интеллекта» |
| Дата | 7 октября 2026 г. |

> **Суть.** ИИ-агент работает на одноплатном компьютере, к которому подключены датчики, камера, исполнительные устройства или мобильный робот. «Думает» внешняя языковая модель (GPT mini/nano через RouterAI), а сведения об оборудовании агент получает через MCP-серверы. Агент ведёт два контекста — пользователя и оборудования — и сам решает, когда и с какой справкой обращаться к модели.
>
> **Главный критерий** (как предложено на встрече) — не надёжность, а **работа системы во времени, её производительность и экономность**. За рабочий день измеряются вызовы модели, токены, затраты в рублях, трафик, память и загрузка процессора платы, задержка реакции; отдельно проверяется работа при предельно жёстких ограничениях ресурсов.

## Почему мой подход хорош именно для одноплатника

| Что | Как устроено | Что уже видно (предварительно) |
|---|---|---|
| Лёгкий агент | Один статический исполняемый файл на Go без зависимостей: не нужны ни Node.js, ни Python, ни контейнеры | в простое около 13 МиБ памяти в виртуальной плате; около 26–29 МиБ при проигрывании рабочего дня (на ноутбуке, RSS). Статическая сборка: по требованиям Go должна запускаться и на старой системе Jetson Nano (Ubuntu 18.04, ядро 4.9), где OpenClaw и NanoClaw штатно не работают; на самой плате пока не проверялась |
| Модель вызывается, только когда нужно | Событийный фильтр и локальные правила на плате вместо «проверки по таймеру» раз в 30 минут | ожидается меньше вызовов, токенов и пробуждений процессора; прямого замера «таймер против фильтра» ещё нет (первые ускоренные прогоны испортила ошибка стыка агента и эмулятора помещения); в пробном сравнении с аналогами — 77 вызовов против 90–102 |
| Короткий контекст | Стабильное начало запроса в расчёте на кэш провайдера, компактный каталог инструментов, состояние оборудования по требованию, сокращение истории | около 2,7 тыс. входных токенов на один вызов модели при ответе на реплику против 8,1–8,3 тыс. у PicoClaw (профиль с seahorse) и 10,8–11,5 тыс. у nanobot; по сумме входных токенов за фрагмент дня против самой экономной найденной настройки PicoClaw — примерно в 1,9 раза меньше |

| | | |
|---|---|---|
| Робот: сводки вместо сырых данных | Свой MCP-адаптер робота: 5 инструментов, сводки и события «доехал / прервано / препятствие», кадр только по запросу | в 3,7–10 раз меньше входных токенов на задачу и примерно в 110–220 раз меньше трафика на поездку, чем у готового ros-mcp-server; память 11–15 МиБ против ≈110 МиБ |
| Камера на плате (планируется) | Детектор объектов будет работать на плате, в модель уйдёт текстовая сводка, а не кадр; сервер камеры ещё не написан | по токенам кадр дороже сводки в несколько раз, а у некоторых моделей — в сотни раз |
| Безопасность приводов почти без затрат | Детерминированная проверка каждой команды в MCP-сервере, а не правила в запросе к модели | около 0,5 мкс на проверку |
| Предельные ограничения | «Лестница ограничений» на виртуальной плате: память до 32 МиБ, процессор до 10 % ядра, плохая сеть, медленная карта памяти | лимиты памяти и процессора проверены (при 32 МиБ процесс, которому нужно 64 МиБ, завершается; квота 10 % даёт 0,3 с процессора за 3 с), средства для сети и диска (tc netem, io.max) доступны; «карта выживаемости» агентов — следующий этап |

Все числа — предварительные: прогоны шли на модели-заглушке (без реального API) и при параллельной нагрузке на ноутбук. Итоговые замеры будут сделаны по протоколу на свободной машине, на реальной модели и на плате. Сравнение с аналогами прошло независимое ревью. Оно нашло перекосы в пользу моего агента: неудачную настройку PicoClaw, сжатие времени у nanobot и разную основу сравнения памяти. В перепрогоне ревьюера с более экономной настройкой PicoClaw разрыв по входным токенам сократился с 3,1 до 1,9 раза; стенд сравнения исправляется, после этого числа будут получены заново.

## Что я посмотрел

Просмотрел несколько сотен научных работ и проектов; ключевые источники проверены по первоисточнику (список — `research/new_topic/SOURCES.md`). Существующие агенты делятся на несколько классов: личные агенты на собственном оборудовании пользователя (OpenClaw и его облегчённые варианты NanoClaw, PicoClaw, nanobot), умный дом (Home Assistant с языковой моделью и MCP), агенты для роботов на ROS (ROSA, ros-mcp-server), универсальные фреймворки и научные системы. Схема «тонкий агент + внешняя модель + MCP к оборудованию» уже стала распространённой, поэтому новизна не в ней. **Пробел**: эти агенты меряли на коротких задачах. Измерений того, как они ведут себя в течение рабочего дня на одноплатнике и при жёстких ограничениях, в найденных источниках нет. Подробно — в документе «Что я изучил».

**Что в работе моё.** (1) Измеренный профиль «рабочего дня» агента на одноплатнике — вызовы, токены, рубли, трафик, память, процессор — и условия, при которых каждый приём экономии окупается. (2) Агент, который подписывается на изменения оборудования по MCP: у PicoClaw и nanobot MCP-клиент берёт у сервера только инструменты, события датчиков до них сами не доходят. (3) MCP-сервер оборудования, который сам строит описание датчиков и реле по данным ядра Linux (готовой реализации в проверенных источниках не найдено). (4) Компактный адаптер робота: сводки и события вместо сырых потоков.

## Что уже сделано

- Работающий прототип системы: агент на Go, MCP-серверы датчиков и реле, адаптер робота, генератор «рабочего дня» по записям датчиков реальных домов (набор CASAS) и данным исследований использования голосовых ассистентов, эмулятор помещения, модель-заглушка для бесплатных прогонов, метрики и статистика.
- Стенд: виртуальная «плата» (ARM, 4 ядра, 4 ГБ памяти) с регулируемыми ограничениями и виртуальная машина-симулятор робота: ROS 2, Gazebo, TurtleBot3, навигация Nav2. С «платы» робот управляется по сети через мой MCP-адаптер (пока тестовым MCP-клиентом; подключение самого агента — следующий шаг).
- Установлены и подключены к тому же стенду аналоги для сравнения: PicoClaw и nanobot. NanoClaw в замерах не участвует: внутри у него Claude Agent SDK, с GPT mini через RouterAI он штатно не работает, нужны Docker и Node.js ⩾ 22; из него взят приём «скрипт решает, будить ли модель».
- Серверы оборудования и робота, стенд «рабочего дня», сэмплер и обвязка аналогов прошли независимое ревью с замерами; ревью самого агента было прервано и будет повторено. Найденные ошибки исправляются. Подробно — в документе «Что уже сделано и первые замеры».

## Что прошу и что хочу согласовать

1. **Доступ к RouterAI** (командный аккаунт с лимитом): по расчёту основные эксперименты стоят около 1,3–1,5 тыс. ₽ при gpt-4o-mini, с запасом ×1,5 — около 1,9–2,3 тыс. ₽ (если основной моделью станет gpt-5-mini — около 3,8 тыс. ₽).
2. **Доступ к Jetson Nano** на кафедре и уточнение модификации платы (4 ГБ или 2 ГБ) — для переноса с виртуальной платы и реальных 8-часовых прогонов.
3. При возможности — **периферия**: USB-камера, датчик температуры, датчик движения, реле.
4. **Согласовать** два сценария (помещение с датчиками и мобильный робот) и формулировку цели (как в плане ВКР): сократить суточные затраты на обращения к внешней модели (вызовы, входные токены, рубли) относительно периодической фоновой проверки — штатного режима аналогов, а относительно PicoClaw в лучшей штатной настройке — не превысить их; при этом полнота реакции на значимые события не хуже (допуск 5 п. п.), небезопасных команд — ноль.